

Surveys of Algebraic and Probabilistic Perspectives on the Rubik's Cube
MATH 441

Rochester Institute of Technology
Ducheng Tan

May 16, 2026

Abstract

One of the most well-known puzzles, the $3 \times 3 \times 3$ Rubik's Cube, with a unique color on each face, can be analyzed as a group $\mathcal{R}$ that embeds naturally into the symmetric group S_{48} . In this paper we introduce the Rubik's Cube Group $\mathcal{R} = \langle R, L, F, B, D, U \rangle$ and explore its structure using elementary group theory. We also examine the Cayley graph of the cube, discuss the role of vertex transitivity in bounding the diameter (God's number), and present a statistical method that yields a highconfidence upper bound for the diameter with only modest computation. A brief numerical implementation in python will be demonstrated. This paper is a brief overview of *A Demigod's Number for the Rubik's Cube* [1] and *The diameter of the rubik's cube group is twenty* [2] with some unique insights gained from taking a first course in Abstract Algebra I from Professor Raymond Jones at RIT.

1 Introduction

The $3 \times 3 \times 3$ Rubik's Cube is a rich topic of combinatorial and algebraic problems, that is often not discussed in a regular college course. Initially, it is easy to see that labeling each of the 54 colored stickers and observing that any face turn permutes these stickers, we can regard each state of the cube as a permutation of a set of labels. Because the six center pieces never move relative to one another, we may ignore them and work with the 48 movable stickers. Thus every legal cube state corresponds to an element of the symmetric group S_{48} .

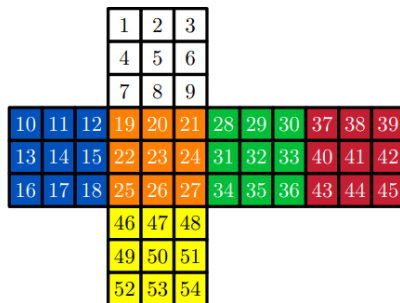


Figure 1: S_{48} labels. *Top face:* 1, 2, 3, 4, (5 = center, ignored), 6, 7, 8, 9 *Front face:* 10, 11, 12, 13, (14 = center, ignored), and so on [1]

A single move say a 90° clockwise turn of the in the perspective of any face acts as a specific permutation on the 48 stickers. Denote the set of basic moves by the standard cubing community's notation

$$G_m = \{R, L, F, B, D, U\},$$

where each letter stands for a clockwise quarterturn of the corresponding face. The inverse moves (counter-clockwise) and double turns are obtained by repeating or inverting these generators. Let s be a state of the cube then applying a move $M \in G_m$ produces a new state $M(s)$. Since every move is a bijection on the set of states, we have

$$M : \mathcal{R} \rightarrow \mathcal{R}, \quad M(s) \in \mathcal{R} \text{ for all } s \in \mathcal{R}.$$

The collection of all states reachable from the solved configuration forms a subgroup of S_{48} , which we call the *Rubik's Cube group* and we denote the basic moves as the generator of $\mathcal{R}$

$$\mathcal{R} = \langle R, L, F, B, D, U \rangle.$$

Corollary 1.1. (*Cayley's Theorem*) *Every group is isomorphic to a subgroup of some symmetric group. If G is a group of order n , then G is isomorphic to a subgroup of S_n [3]*

This fundamental result tells us that $\mathcal{R} \cong G$ where G is a subgroup of $S_{|\mathcal{R}|}$. We write $G \leq S_{|\mathcal{R}|}$. Note that the squares of a Rubik's cube (which we can call cubies) can be arranged in such way, 8 corner pieces can be arranged in $8!$ different ways, each of which can be arranged in 3 different orientations. This means that there are 3^8 possible ways to organize the corner pieces. Then, there are $12!$ different ways to arrange the 12 edge pieces. Since each edge has 2 alternative orientations, there are then 2^{12} possible arrangements for each permutation of edge pieces. However, only $1/3$ of the Rubik's cube permutations have the corner cubies' rotations properly. Just $1/2$ of the permutations have the proper cubie-rearrangement parity and the same edge-flipping orientation as the original cube. This count gives

$$|\mathcal{R}| = \frac{8! \cdot 3^8 \cdot 12! \cdot 2^{12}}{3 \cdot 2 \cdot 2} = 43252003274489856000$$

However we know that $|\mathcal{R}|$ as a order of a group is so large that it is difficult to extract meaningful insights from. Thus we turn to a tighter perspective of $\mathcal{R} \leq S_{48}$ for reasons stated at first.

1.1 Group Structure of $\mathcal{R}$

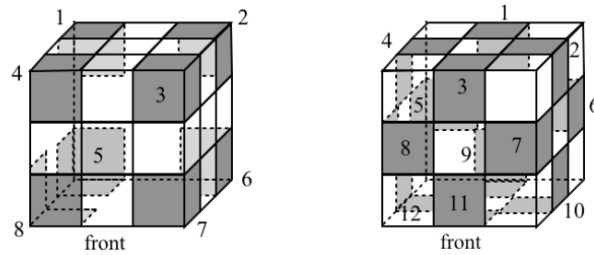


Figure 2: Edge and Corner Labeling for ρ, σ [4]

An alternate way to think about $\mathcal{R}$ is by looking at the product of symmetry groups of each type of cubies on the cube. If X_s is an arbitrary state of the cube, then the position vector can be seen as a 4-tuple $(\rho, \sigma, \mathbf{v}, \mathbf{w})$, where $\rho \in S_8, \sigma \in S_{12}, \mathbf{v} \in \mathbb{Z}_3^8, \mathbf{w} \in \mathbb{Z}_2^{12}$. More explicitly, $\rho(i) = j$ if corner cubie i is in the labeled position cubicle, $\sigma(i) = j$ if an edge i is permuted to position j . In the vector $\mathbf{v}$ each v_i is the number that represents the orientation of the i th corner cubie with 3 possible states $\{0, 1, 2\} = \mathbb{Z}_3$. The same applies for the vector $\mathbf{w}$ both for the possible states of the edges with each cubie w_i labeled as $\{0, 1\} = \mathbb{Z}_2$.

Theorem 1.2. (*The Fundamental Theorem of Cubology*). *A position vector $(\rho, \sigma, \mathbf{v}, \mathbf{w}) \in S_8 \oplus S_{12} \oplus \mathbb{Z}_3^8 \oplus \mathbb{Z}_2^{12}$ corresponds to a legal configuration of the Rubik's cube if and only if the following three conditions are satisfied.*

- $\text{sign}(\rho) = \text{sign}(\sigma)$
- $\sum_{1 \leq i \leq 8} v_i \equiv 0 \pmod{3}$
- $\sum_{1 \leq i \leq 12} w_i \equiv 0 \pmod{2}$

These constraints eliminate the non-physical and purity cases of the cube. Taking a detailed look at the homomorphisms under the structure of $\mathcal{R}$. Consider the corner orientation. Let $\mathcal{C} \leq \mathcal{R}$ such that this subgroup only contains the permutations that affect only the orientation of corners. There exist a homomorphism

$$\phi_{\mathcal{C}} : \mathcal{R} \rightarrow \bigoplus_{i=1}^7 \mathbb{Z}_3 \cong \mathbb{Z}_3^7$$

that records the twist of seven independent corners and we know that the eighth is determined by the zero-sum condition $\sum_{1 \leq i \leq 8} v_i \equiv 0 \pmod{3}$ stated in Theorem 1.2. The kernel of $\phi_{\mathcal{C}}$ consists precisely of those states in which all corners are correctly oriented. By the First Isomorphism Theorem,

$$\mathcal{R} / \ker(\phi_{\mathcal{C}}) \cong \text{im}(\phi_{\mathcal{C}}) = \mathbb{Z}_3^7.$$

Thus there are exactly $3^7 = 2187$ possible corner-orientation patterns. Similarly, for edge flips we have a homomorphism

$$\phi_e : \mathcal{R} \rightarrow \mathbb{Z}_2^{11}$$

whose image is isomorphic to $\mathbb{Z}_2^{11}$, giving $2^{11} = 2048$ edge-orientation patterns. The parity constraint is captured by the sign homomorphism from the permutation of corners and edges, forcing the product of their signs to be $+1$.

1.2 Cayley Graphs

Given a group G and a generating set $S \subseteq G$, the *Cayley graph* $\Gamma(G, S)$ is the graph whose vertices are the elements of G , and where two vertices $g, h \in G$ are joined by an edge if there exists a generator $s \in S$ such that $gs = h$. For the Rubik's Cube, we take the generating set

$$S_{\mathcal{R}} = \{R, L, U, D, F, B\},$$

which consists of the 6 distinct face turns in the half-turn metric. The vertices of $\Gamma(\mathcal{R}, S_{\mathcal{R}})$ are the $\approx 4.3 \times 10^{19}$ states of the cube, and edges connect states that differ by a single move. A crucial property of every Cayley graph is that it is *vertex-transitive* meaning that for any two vertices u, v , there exists an automorphism of the graph mapping u to v . In the case of a group, left multiplication by vu^{-1} provides such an automorphism. Vertex transitivity means that the graph will look similar from every vertex; in particular, the local structure (neighborhoods, distances to other vertices) is identical regardless of the starting state. Let $d(u, v)$ denote the distance between states u and v , i.e., the minimum number of moves required to transform u into v . The *diameter* D of the graph is the maximum distance between any pair of vertices, this is exactly what we define as God's number. The *mean distance* μ is the average distance over all pairs of vertices. Since we mention that the vertex transitive, μ can be computed by fixing one vertex (We are interested in the solved state) and averaging its distance to all other vertices.

$$\mu = \frac{1}{|\mathcal{R}| - 1} \sum_{v \in \mathcal{R}} d(\text{solved}, v).$$

The relationship between diameter and mean distance in vertex-transitive graphs is remarkably tight. Using the triangle inequality and symmetry, one can prove the following elementary bound.

Theorem 1.3 (Diameter–Mean Distance Bound). *For any vertex-transitive graph with diameter D and mean distance μ , we have*

$$D < 2\mu.$$

Proof. Let u and v be vertices such that $d(u, v) = D$. By vertex transitivity,

$$\mu = \frac{1}{|V| - 1} \sum_{x \in V} d(u, x) = \frac{1}{|V| - 1} \sum_{x \in V} d(v, x).$$

Adding the two expressions gives

$$2\mu = \frac{1}{|V| - 1} \sum_{x \in V} (d(u, x) + d(v, x)) \geq \frac{1}{|V| - 1} \sum_{x \in V} d(u, v) = \frac{|V|}{|V| - 1} D > D,$$

where the inequality follows from the triangle inequality $d(u, x) + d(x, v) \geq d(u, v)$. □

This theorem provides a true mean of the distance from any state to the solved state, which allows for statistical approach to bounding God's number through sampling. If we can estimate μ from a random sample, then twice that estimate provides an upper bound for D with high probability.

1.3 PLL : T Permutation

As a concrete illustration of permutations within $\mathcal{R}$, we examine a well-known algorithm from the CFOP speed-solving method. The last step of CFOP, called PLL (Permutation of the Last Layer), consists of sequences that permute the pieces of the final layer without disturbing the rest of the cube. We will introduce the *T-Permutation*, which is one such algorithm. Note that there is different sequence of moves for a single PLL permutation as we have shown that there is different paths on the graph to get to the same state. In this example, we will use the standard move of the T Permutation as

$$R U R' U' R' F R^2 U' R' U' R U R' F'.$$

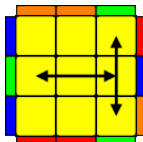


Figure 3: T-Permutation [5]

Executing this sequence on a solved cube swaps two adjacent corners and two adjacent edges on the top layer, leaving the remainder of the cube intact. In the language of group theory, the T-Permutation is an element of $\mathcal{R}$ of order 2 (applying it twice returns the cube to the solved state). It can be expressed as a product of the basic generators and their inverses, and as such it corresponds to a specific permutation of the 48 stickers.

Our accompanying Python implementation in Appendix 1.4 defines each basic move as a permutation in S_{48} using the labeling of Figure 1. The program allows the user to input any sequence of moves, including the T-Permutation, and computes the resulting sticker arrangement. It also verifies that applying the T-Permutation twice indeed yields the identity permutation, confirming its order.

1.4 Statistical Validation

The diameter bound $D < 2\mu$ suggests that if we can obtain a reliable estimate of the mean distance μ , we can bound the diameter from above. Computing μ exactly is as hard as knowing the full distance distribution, which is infeasible for a group of size 4.3×10^{19} . However, we can *sample* states uniformly at random and compute or bound their distance to the solved state. Standard concentration inequalities then give a confidence interval for μ .

Merino and Subercaseaux [1] applied Hoeffding’s inequality to this problem. Let S be a set of states sampled uniformly at random, and let $\hat{\mu}_S = \frac{1}{|S|} \sum_{s \in S} d(s)$ be the empirical average distance. Since the distance $d(s)$ is bounded. For instance, by a “human number” of 205 moves using a beginner’s method, Hoeffding’s inequality states that for any $\varepsilon > 0$,

$$\Pr[|\hat{\mu}_S - \mu| \geq \varepsilon] \leq 2 \exp\left(-\frac{2|S|\varepsilon^2}{C^2}\right),$$

where C is the maximum possible distance (here $C = 205$). Choosing $\varepsilon = 0.18$ and using the union bound together with the deterministic inequality $D < 2\mu$ yields

$$\Pr[D \geq 2\hat{\mu}_S + 0.36] < 2 \exp\left(-\frac{|S|}{1\,541\,939}\right).$$

In practice, one can do much better. By using an efficient solver (such as Kociemba’s two-phase algorithm) one can compute the exact distance for each sampled state, and the maximum distance actually observed among millions of random samples never exceeds 20. This allows us to replace the crude bound $C = 205$ with $C = 20$ for the vast majority of samples, dramatically reducing the required sample size. With $|S| = 500\,000$ samples, the empirical mean distance is around 18.32 moves [1], and the probability that the true diameter exceeds 36 becomes astronomically small, e.g. well below 10^{-10} . Thus we obtain a “demigod’s number” of 36 moves with an overwhelming degree of confidence, using only a few hours of computation on a personal computer. Namely for large sample size $|S|$ we can use

$$\hat{\mu}_s \sim N\left(\mu, \frac{\sigma^2}{|S|}\right)$$

to construct confidence intervals and make inferences about μ_s . Merino and Subercaseaux’s paper implemented a solver to obtain the results stated above. [SOLVER LINK](#) In addition we can show that $\hat{\mu}_S$ is an unbiased estimator. We observe $n = |S|$ i.i.d. draws $X_1, \dots, X_n$ where each $X_i = d(s_i)$ is the distance of a uniformly sampled state $s_i \in \mathcal{R}$ to the solved state. This is equivalent to claiming that each scramble is independent from another which is reasonable, and we assume identical distribution to time being. Each X_i takes values in $\{0, 1, \dots, K\}$ where $K = 20$ (God’s number), with an unknown probability mass function (I conjecture that this mass function actually depends on the habit of person scrambling the cube or the psuedo randomness used by computers to generate the scrambler).

$$p_k = \Pr[d(s) = k] = \frac{N_k}{|\mathcal{R}|}, \quad k = 0, 1, \dots, K,$$

where N_k is the number of positions at distance k . The parameter of interest is $\mu = \mathbb{E}[d(s)] = \sum_{k=0}^K k \cdot p_k$. Recall the empirical mean estimator

$$\hat{\mu}_S = \frac{1}{n} \sum_{i=1}^n X_i.$$

Proposition 1.4. $\hat{\mu}_S$ is an **unbiased** estimator of μ .

Proof. Since each X_i is drawn i.i.d. from the same distribution,

$$\mathbb{E}[\hat{\mu}_S] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i] = \frac{1}{n} \cdot n\mu = \mu. \quad \square$$

References

- [1] Arturo Merino and Bernardo Subercaseaux. A demigod’s number for the rubik’s cube. *arXiv preprint arXiv:2501.00144*, 2024.
- [2] Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge. The diameter of the rubik’s cube group is twenty. *siam REVIEW*, 56(4):645–670, 2014.
- [3] D Dummit. Errata abstract algebra, by d. dummit and r. foote (most recently revised on august 15, 2014) these are errata for the third edition of the book. errata from previous editions have been fixed.
- [4] Jamie Mulholland. Math 304: Permutation puzzles — a mathematical perspective, 2021. Department of Mathematics, Simon Fraser University.
- [5] Speedsolving.com Wiki contributors. PLL algorithms – CFOP speedcubing cases. <https://www.speedsolving.com/wiki/index.php/PLL>, 2026. Speedsolving.com Wiki. Accessed: 2026-04-20.

Appendix

Python Code for Mini Rubik's Cube Simulation

Make sure you input *help* to get started.

```
from sympy.combinatorics import Permutation
import random

MOVES = {
    'R': Permutation(25,27,32,30)(26,29,31,28)(3,38,43,19)(5,36,45,21)(8,33,48,24),
    'L': Permutation(9,11,16,14)(10,13,15,12)(1,17,41,40)(4,20,44,37)(6,22,46,35),
    'U': Permutation(1,3,8,6)(2,5,7,4)(9,33,25,17)(10,34,26,18)(11,35,27,19),
    'D': Permutation(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40),
    'F': Permutation(17,19,24,22)(18,21,23,20)(6,25,43,16)(7,28,42,13)(8,30,41,11),
    'B': Permutation(33,35,40,38)(34,37,39,36)(3,9,46,32)(2,12,47,29)(1,14,48,27),
}

MOVES |= {f"{k}'": v**-1 for k, v in MOVES.items()} | {f"{k}2": v**2 for k, v in MOVES.items()}

PLL = {
    "T": "R U R' U' R' F R2 U' R' U' R U R' F'",
    "Y": "F R U' R' U' R U R' F' R U R' U' R' F R F'",
}

IDENTITY = Permutation(list(range(49)))

def parse_alg(alg):
    result = IDENTITY
    for move in alg.strip().split():
        result = result * MOVES[move]
    return result

def apply(state, alg):
    return parse_alg(alg)(state)

HELP = """
Commands:
moves <seq>      Apply move sequence (e.g. moves R U R' U')
pll <name>        Apply PLL algorithm
scramble [n]      Scramble with n random moves (default 20)
reset            Reset to solved state
solved           Check if solved
state [n]         Show first n stickers (default 20)
pll_list         List PLL algorithms
moves_list       List basic moves
quit            Exit
"""

def main():
    cube = list(range(49))
    print("Rubik's Cube Simulator | type 'help' for commands")

    while True:
        try:
            parts = input("\n> ").strip().split(maxsplit=1)
            if not parts:
                continue
            cmd, arg = parts[0].lower(), parts[1] if len(parts) > 1 else ""

            if cmd in ("quit", "exit"):
                print("Goodbye!"); break
```

```

elif cmd == "help":
    print(HELP)
elif cmd == "reset":
    cube = list(range(49)); print("Reset to solved.")
elif cmd == "solved":
    print("SOLVED" if cube == list(range(49)) else "NOT solved")
elif cmd == "state":
    n = int(arg) if arg.isdigit() else 20
    print(f"State (first {n}): {cube[:n]}")
elif cmd == "pll_list":
    print("PLLs:", " ", ".join(PLL))
elif cmd == "moves_list":
    print("Moves:", " ", ".join(MOVES))
elif cmd == "pll":
    name = arg.upper()
    if name in PLL:
        cube = apply(cube, PLL[name]); print(f"Applied {name}-Perm. Solved? {cube == list(range(49))}")
    else:
        print(f"Unknown PLL '{name}'. Use pll_list.")
elif cmd == "moves":
    if arg:
        cube = apply(cube, arg); print(f"Applied: {arg}. Solved? {cube == list(range(49))}")
    else:
        print("Specify moves.")
elif cmd == "scramble":
    n = int(arg) if arg.isdigit() else 20
    for _ in range(n):
        cube = apply(cube, random.choice(list(MOVES)))
    print(f"Scrambled with {n} moves. Solved? {cube == list(range(49))}")
else:
    print(f"Unknown command '{cmd}'. Type 'help'.")

except KeyboardInterrupt:
    print("\nGoodbye!"); break
except Exception as e:
    print(f"Error: {e}")

if __name__ == "__main__":
    main()

```